

Project Report

Vulnerability Trend and Pattern Analysis Using CVE Vulnerability Dataset

1. Introduction

Cybersecurity vulnerability databases contain a large amount of information about software vulnerabilities, their severity, affected products, weakness categories, and publication history. Analyzing this information can help identify common vulnerability patterns and understand how cybersecurity vulnerabilities change over time.

This project focuses on performing **Vulnerability Trend and Pattern Analysis** using a cleaned CVE vulnerability dataset collected from the **Open Source Vulnerability (OSV)** database. The project uses Python, Pandas, Matplotlib, and Scikit-learn to perform exploratory data analysis, statistical analysis, visualization, correlation analysis, and K-Means clustering.

The main goal is to transform the collected vulnerability records into meaningful information that can be used to understand vulnerability trends, severity patterns, affected software ecosystems, common CWE categories, and groups of similar vulnerabilities.

2. Project Objective

The primary objective of this project is to analyze historical vulnerability data and identify important trends and patterns in cybersecurity vulnerabilities.

The project specifically aims to:

- Analyze the number of vulnerabilities reported in different years.
 - Study the distribution of vulnerability severity levels.
 - Analyze CVSS score statistics.
 - Identify the most frequently occurring CWE categories.
 - Identify commonly affected software ecosystems.
 - Identify frequently affected software packages.
 - Analyze changes in average CVSS scores over time.
 - Study relationships between numerical dataset attributes.
 - Group similar vulnerabilities using K-Means clustering.
 - Generate charts and tables that make the findings easy to understand.
-

3. Data Source

The dataset used in this project is collected from **OSV.dev**, an open-source vulnerability database that provides information about known software vulnerabilities.

The original data was collected through Python-based web scraping and stored in CSV format. After collection, the dataset was cleaned by removing duplicate records, handling empty rows, standardizing text values, and converting important fields into suitable data types.

Primary Data Source:

<https://osv.dev/>

Additional reference sources include:

<https://www.cve.org/>

<https://pandas.pydata.org/>

<https://matplotlib.org/>

<https://scikit-learn.org/>

4. Dataset Description

The dataset contains information related to CVE vulnerabilities.

Attribute	Type	Description
CVE ID	String	Unique identifier of the vulnerability
Severity	String	Severity category such as Critical, High, Medium, or Low
CVSS Score	Float	Numerical severity score of the vulnerability
Published	DateTime	Date when the vulnerability was published
Modified	DateTime	Date when the vulnerability information was modified
Summary	Text	Short description of the vulnerability
Details	Text	Detailed vulnerability information
Ecosystem	String	Software ecosystem affected by the vulnerability
Package	String	Affected software package or project
CWE	String	Common Weakness Enumeration category
References	Text	Security-related reference information

5. Technologies and Libraries Used

The project is implemented using Python.

The main technologies and libraries are:

- **Python** – Main programming language.
- **Pandas** – Data loading, cleaning, grouping, aggregation, and statistical analysis.
- **NumPy** – Numerical calculations and data processing.
- **Matplotlib** – Generation of charts and visualizations.
- **Scikit-learn** – K-Means clustering and feature standardization.
- **CSV** – Storage format for the vulnerability dataset.

The project is executed inside a Python virtual environment to keep the project dependencies isolated from the operating system.

6. Data Preparation and Cleaning

Before performing the analysis, the collected dataset is prepared and cleaned.

The cleaning process includes removing completely empty rows and duplicate records. Text fields are stripped of unnecessary whitespace and standardized where required. CVE IDs are checked against the expected CVE format.

Important fields are also converted into suitable data types. Publication dates are converted into datetime format so that the publication year can be extracted. CVSS scores are converted into numerical values so that statistical calculations can be performed.

Additional analysis features are created from the existing dataset. For example, the publication year is extracted from the published date, the length of vulnerability details is calculated, and the number of references associated with each vulnerability is counted.

This prepared dataset is then used as the input for the Data Insight stage.

7. Methodology

The complete project follows the workflow:

Collected Dataset → Data Cleaning → Feature Preparation → Exploratory Data Analysis → Statistical Analysis → Visualization → Correlation Analysis → K-Means Clustering → Interpretation

The analysis is divided into several major stages.

7.1 Vulnerability Severity Analysis

The dataset is grouped according to severity categories such as Critical, High, Medium, and Low.

The total number of records belonging to each severity category is calculated. A bar chart and pie chart are generated to provide a visual representation of the severity distribution.

This analysis helps determine which severity categories occur most frequently in the collected dataset.

7.2 Year-wise Vulnerability Analysis

The publication date is converted into a year value. Vulnerabilities are then grouped by publication year.

The number of vulnerabilities reported during each year is calculated and visualized using a bar chart.

This makes it possible to observe changes in the number of collected vulnerabilities over the analyzed period.

7.3 CVSS Score Analysis

CVSS scores are analyzed using basic statistical measures such as minimum, maximum, average, and standard deviation.

A histogram is generated to show the distribution of CVSS scores.

The average CVSS score for each publication year is also calculated. A line chart is used to visualize changes in average CVSS scores over time.

7.4 CWE Analysis

CWE identifiers are extracted from the dataset and counted to determine the most frequently occurring weakness categories.

The most common CWE categories are displayed using a bar chart. This analysis helps identify the types of software weaknesses that appear most often in the collected vulnerability records.

7.5 Ecosystem Analysis

Vulnerabilities are grouped according to their affected software ecosystem.

The analysis identifies the most common ecosystems represented in the dataset and produces a visualization showing their frequency.

This provides an overview of which software development ecosystems are most represented in the collected records.

7.6 Package Analysis

The affected package field is analyzed to identify packages that appear frequently in the vulnerability dataset.

The top affected packages are displayed using a horizontal bar chart. This helps identify software projects that appear repeatedly within the collected vulnerability records.

7.7 Correlation Analysis

Several numerical attributes are examined using correlation analysis.

The project considers numerical fields such as:

- CVSS score
- Publication year
- Number of references
- Vulnerability detail length

A correlation matrix is generated and exported as a CSV file. The matrix can be used to examine whether numerical features have positive, negative, or weak relationships with each other.

Correlation represents association between numerical variables and does not by itself establish causation.

8. K-Means Clustering

In addition to exploratory analysis, the project uses **K-Means clustering** to group vulnerabilities with similar numerical characteristics.

The clustering process uses selected numerical features such as:

- CVSS score
- Publication year
- Number of references
- Vulnerability detail length

Because these attributes have different numerical scales, the features are standardized using **StandardScaler** before clustering.

K-Means is then applied to divide the vulnerabilities into a small number of clusters. The project uses a simple configuration suitable for an academic exploratory analysis.

The clustering results include:

- Cluster assignment for analyzed vulnerabilities.
- Number of vulnerabilities in each cluster.

- Average feature values for each cluster.
- A chart showing cluster sizes.

The purpose of clustering is not to assign security severity or predict future vulnerabilities. Instead, it is used to identify groups of records that have similar numerical characteristics.

9. Data Visualizations

The project automatically generates several charts to present the findings clearly.

The main visualizations include:

1. **Vulnerability Distribution by Severity**

Shows the number of Critical, High, Medium, Low, and other severity categories.

2. **Vulnerabilities by Publication Year**

Shows how many vulnerabilities were recorded in each publication year.

3. **Average CVSS Score by Year**

Shows how the average CVSS score changes across publication years.

4. **CVSS Score Distribution**

Shows the frequency distribution of numerical CVSS scores.

5. **Top CWE Categories**

Shows the most frequently occurring CWE classifications.

6. **Affected Ecosystems**

Shows the software ecosystems represented most frequently.

7. **Top Affected Packages**

Shows packages occurring most frequently in the dataset.

8. **K-Means Cluster Sizes**

Shows the number of vulnerabilities assigned to each cluster.

These visualizations make the statistical findings easier to understand and present.

10. Expected Data Insights

After running the analysis script on the cleaned dataset, the project will provide information about several important cybersecurity patterns.

The analysis will identify the most common severity category, calculate the average and maximum CVSS scores, identify the year with the highest number of collected vulnerability records, and determine the most frequently occurring CWE categories.

It will also identify which software ecosystems and packages occur most frequently.

The K-Means analysis will provide additional information about groups of vulnerabilities that share similar numerical characteristics.

The actual numerical values will depend on the final cleaned dataset used for the project.

11. Project Outputs

After executing the analysis script, the project generates two major categories of outputs.

Charts

The `outputs/charts/` directory contains the generated visualizations, including:

```
severity_distribution.png
severity_distribution_pie.png
vulnerabilities_by_year.png
average_cvss_by_year.png
cvss_distribution.png
ecosystem_distribution.png
top_packages.png
top_cwe_categories.png
kmeans_cluster_sizes.png
```

Analysis Tables

The `outputs/tables/` directory contains statistical and analytical results such as:

```
dataset_summary.csv
severity_distribution.csv
vulnerabilities_by_year.csv
average_cvss_by_year.csv
cvss_statistics.csv
ecosystem_distribution.csv
top_packages.csv
top_cwe_categories.csv
numeric_correlation_matrix.csv
```

```
kmeans_cluster_summary.csv  
cve_cluster_assignments.csv
```

A human-readable summary of the major findings is also generated as:

```
outputs/insights.txt
```

12. Dataset Quality and Limitations

The analysis depends on the quality and completeness of the collected OSV records. Not every CVE necessarily contains complete metadata, and some vulnerability records may not be available through the OSV web interface.

During the original collection process, network failures, unavailable records, and server-side restrictions may result in some CVEs being skipped. Therefore, the final dataset represents the successfully collected records rather than the complete global CVE population.

Missing CVSS values, publication dates, CWE information, ecosystems, or package information can also affect specific analyses.

K-Means clustering is also sensitive to the selected features and preprocessing method. The resulting clusters should therefore be interpreted as exploratory groups rather than definitive categories of vulnerabilities.

13. Expected Benefits

This project demonstrates how cybersecurity vulnerability data can be transformed into useful analytical information.

The analysis can help:

- Understand historical vulnerability reporting patterns.
- Identify frequently occurring weakness categories.
- Observe severity and CVSS score distributions.
- Identify commonly affected ecosystems and packages.
- Discover relationships between numerical attributes.
- Group similar vulnerability records using clustering.
- Present cybersecurity data using meaningful visualizations.

The project also demonstrates the practical use of data science techniques in the cybersecurity domain.

14. Conclusion

The **Vulnerability Trend and Pattern Analysis** project provides a complete exploratory data analysis workflow for a real-world cybersecurity dataset.

Starting from the cleaned CVE/OSV dataset, the project performs statistical analysis, frequency analysis, time-based analysis, correlation analysis, visualization, and K-Means clustering.

The final outputs provide a structured view of vulnerability severity, CVSS scores, publication trends, CWE categories, affected ecosystems, affected packages, and numerical vulnerability patterns.

Overall, the project demonstrates how Python-based data science techniques can be applied to cybersecurity data to discover trends, summarize large datasets, and identify groups of similar vulnerability records.

15. References

<https://osv.dev/>

<https://www.cve.org/>

<https://pandas.pydata.org/>

<https://numpy.org/>

<https://matplotlib.org/>

<https://scikit-learn.org/>